Software metrics

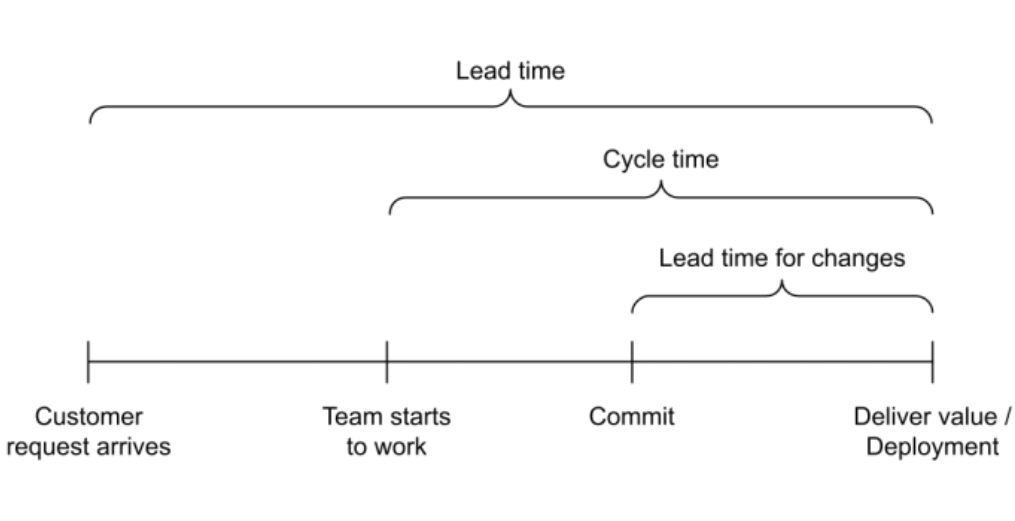
Delivery Metrics

Deployment Frequency: Measures how often updates are released to production.

Lead Time: The total time from request to full deployment of a product or feature, encompassing the entire development and delivery workflow. A key indicator of organizational efficiency.

Cycle Time: The time it takes to complete a specific part of the workflow, from development start to task completion. This metric highlights team productivity and potential bottlenecks.

Lead Time for Changes: Measures the time from a code commit to deployment in production. Short lead times indicate efficient automated workflows.


Lead time
Cycle time
Lead time for changes
Customer request arrives
Team starts to work
Commit
Deliver value / Deployment

Stability Metrics

Change Failure Rate: Tracks the percentage of deployments that need immediate fixes or rollbacks.

Mean Time Between Failures (MTBF): The average time between successive system failures. A high MTBF suggests system stability, while low MTBF calls for improvements in reliability and fault tolerance.

Mean Time to Detect (MTTD): The average time to identify a failure, performance issue, or security problem. Low MTTD ensures quick response to minimize impact on users and business.

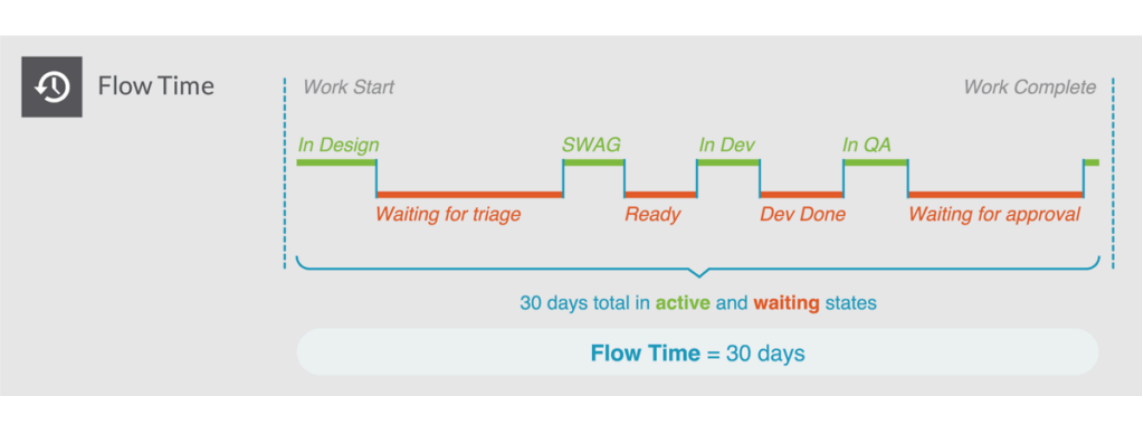
Mean Time to Recovery (MTTR): The average time needed to restore a service after a failure. A low MTTR reflects efficient incident response, with practices like continuous monitoring and automated recovery.

Flow Metrics

Flow Metrics (introduced by the *Flow Framework* by Mik Kersten) help teams understand the **efficiency of the software delivery process itself** — identifying bottlenecks, handoff delays, and waste in the value stream. They measure how work items (features, bugs, risks, technical debts) flow from idea to production. The main Flow Metrics are:

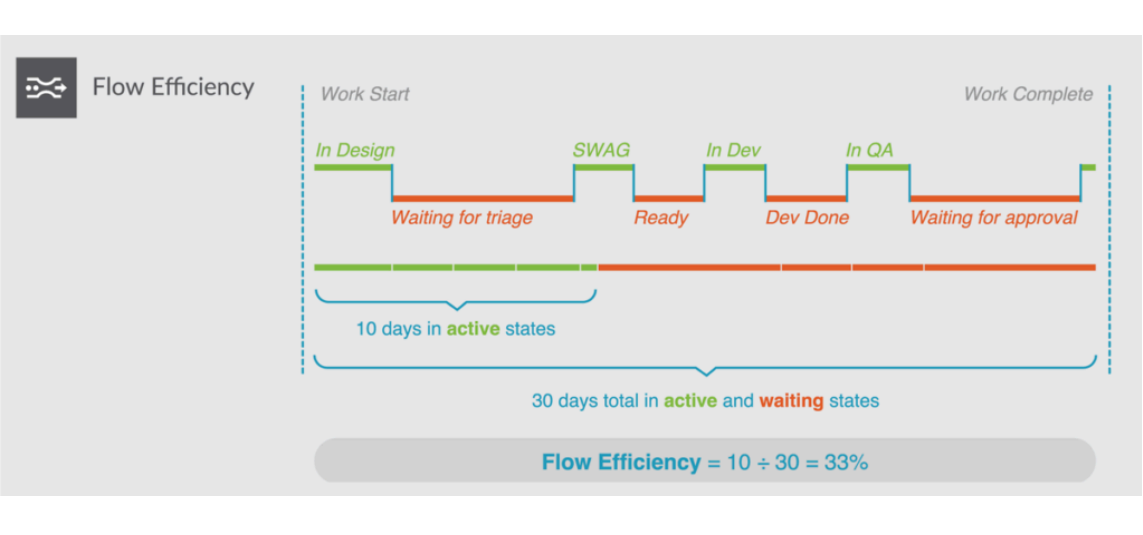
Flow Time: Measures the total time it takes for a work item to go from "start" (when development begins) to "finish" (when value is delivered to users).

- Indicates how long customers wait for value.
- Related to DORA's *Cycle Time*, but considers the *entire value stream*, not just code-to-deploy.


Flow Time
Work Start
Work Complete
In Design
SWAG
In Dev
In QA
Waiting for triage
Ready
Dev Done
Waiting for approval
30 days total in active and waiting states
Flow Time = 30 days

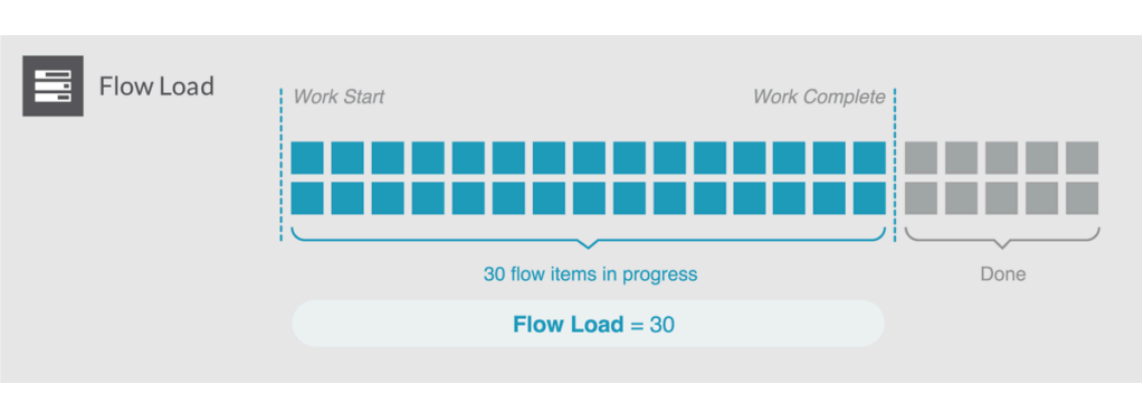
Flow Efficiency: The ratio between *active time* (when work is progressing) and *total flow time* (including waiting).

- Highlights process inefficiencies, approval delays, and overburdened teams.
- High variability or low efficiency signals bottlenecks or context switching.


Flow Efficiency
Work Start
Work Complete
In Design
SWAG
In Dev
In QA
Waiting for triage
Ready
Dev Done
Waiting for approval
10 days in active states
30 days total in active and waiting states
Flow Efficiency = 10 ÷ 30 = 33%

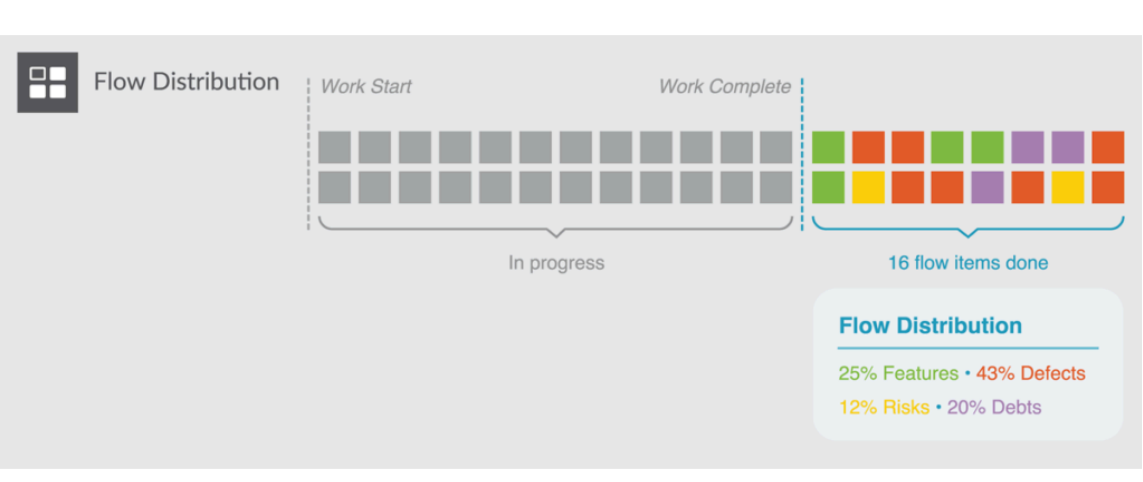
Flow Load: The total number of active work items in progress.

- Helps manage team capacity and prevent overload.
- Strongly correlates with *cycle time* and *quality* — when load is too high, delivery slows and error rates rise.


Flow Load
Work Start
Work Complete
30 flow items in progress
Done
Flow Load = 30

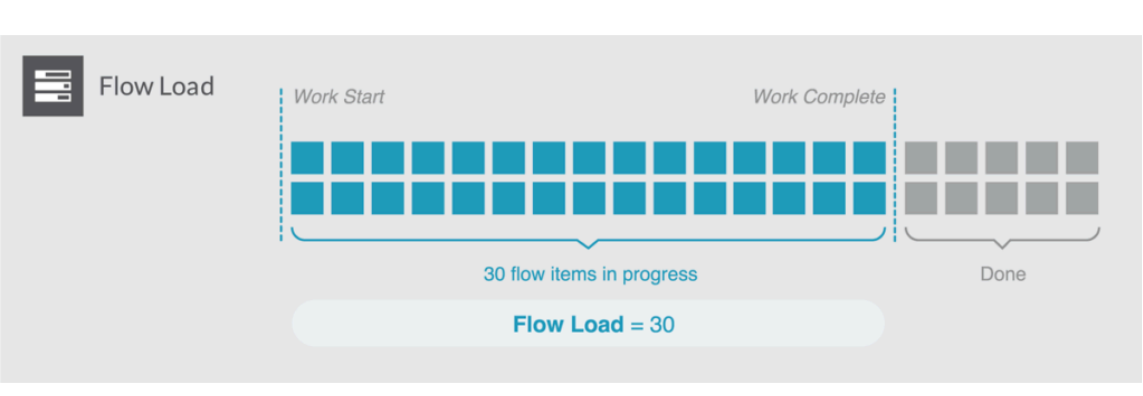
Flow Distribution: Tracks the proportion of different types of work (e.g., features, defects, technical debt, risks).

- Balancing these categories ensures sustainable development — too little time on debt and risk reduction leads to long-term fragility.


Flow Distribution
Work Start
Work Complete
In progress
16 flow items done
Flow Distribution
25% Features • 43% Defects
12% Risks • 20% Debts

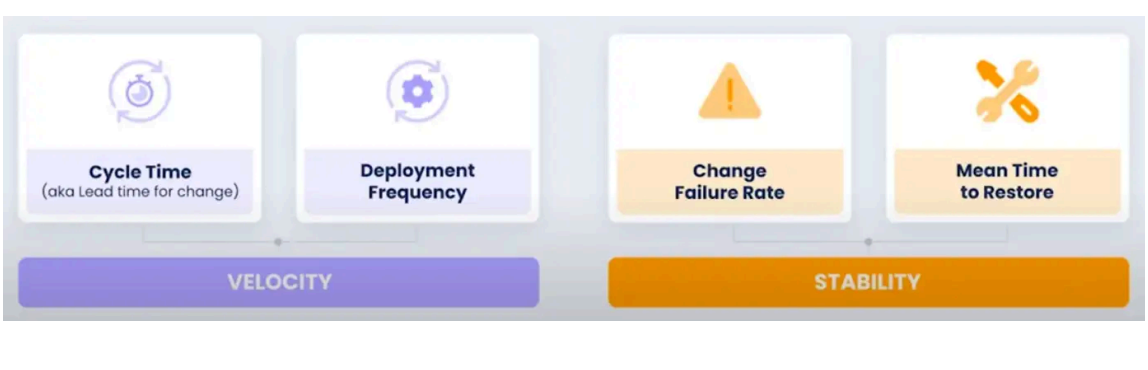
Flow Velocity: The number of work items completed per unit of time.

- Measures delivery throughput.
- Should be analyzed together with Flow Time and Efficiency to avoid focusing solely on output.


Flow Load
Work Start
Work Complete
30 flow items in progress
Done
Flow Load = 30

DORA metrics

The DORA metrics were developed by the DevOps Research and Assessment (DORA) organization, which spent years studying engineering teams and their DevOps processes. They also deliver an interesting annual report.


Cycle Time
(aka Lead time for change)
Deployment Frequency
Change Failure Rate
Mean Time to Restore
VELOCITY
STABILITY

These metrics are valuable because:

- they **correlate with business outcomes** and employee satisfaction, offering industry standards for benchmarking.
- push teams to focus on **continuous improvement**.
- **only four key metrics are needed** to differentiate elite engineering teams from mediocre ones.

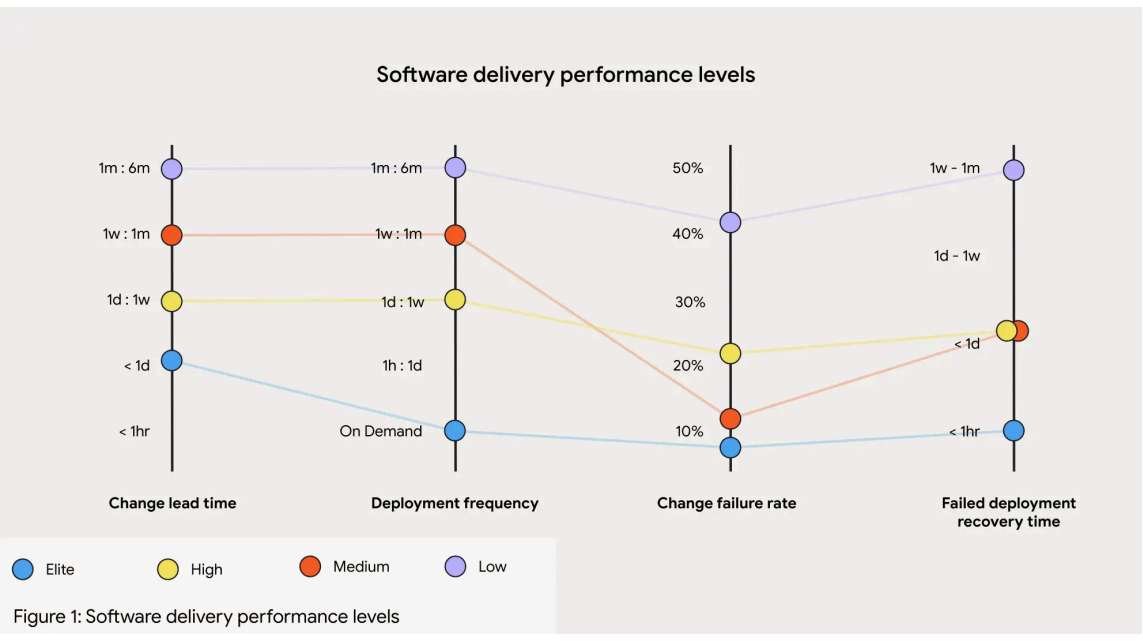
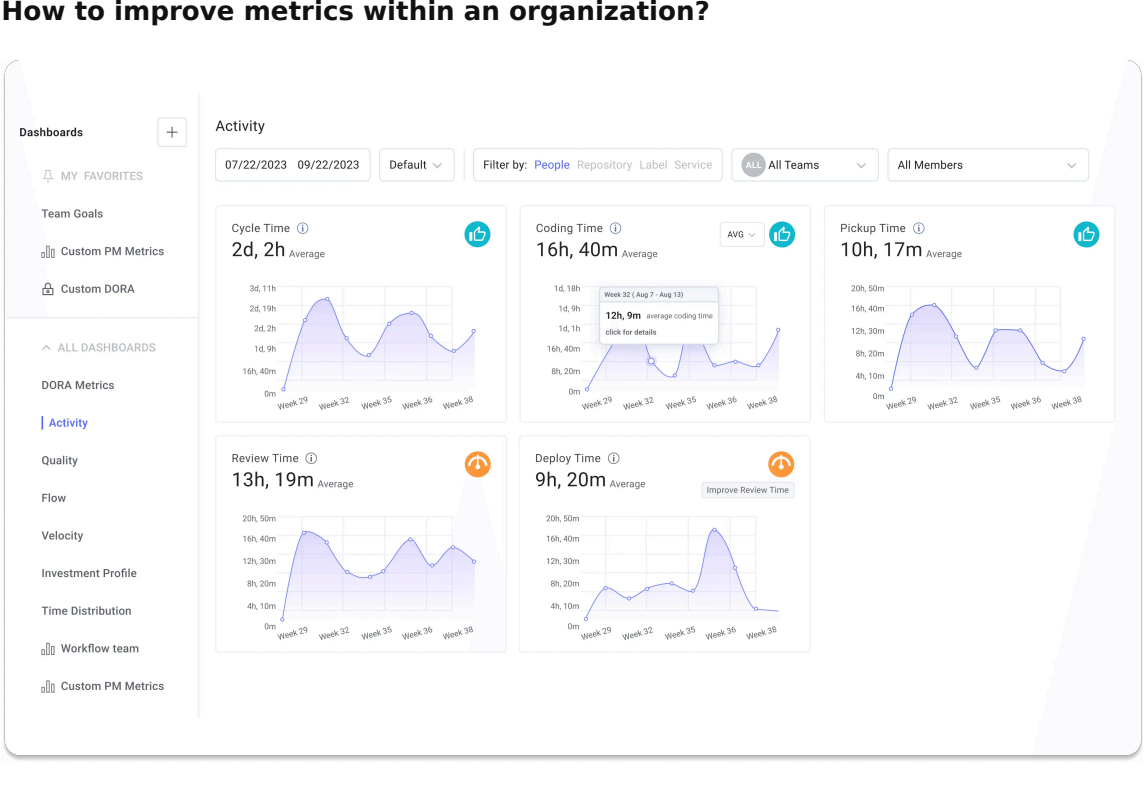


Figure 1: Software delivery performance levels

How to improve metrics within an organization?


Dashboards
MY FAVORITES
Team Goals
Custom PM Metrics
Custom DORA
ALL DASHBOARDS
DORA Metrics
Activity
Quality
Flow
Velocity
Investment Profile
Time Distribution
Workflow team
Custom PM Metrics
Activity
07/22/2023 09/22/2023
Default
Filter by: People Repository Label Service
All Teams
All Members
Cycle Time
2d, 2h Average
Coding Time
16h, 40m Average
AVG
Pickup Time
10h, 17m Average
Review Time
13h, 19m Average
Deploy Time
9h, 20m Average
Improve Review Time

- **Monitor**
- **Reduction of Codebase Size (also PR size)**
- **Automation (CI/CD Pipelines, Deployments Strategies)**

Resources

- https://www.youtube.com/watch?v=H3nlvHQHb5E
- https://www.youtube.com/watch?v=7d5Sah0M6Zg